

Relatório de Métricas de Performance

Comparativo: Paradigma Recursivo × Iterativo

Busca em Profundidade - Hierarquia de Pastas

Contexto do Experimento

O benchmark avaliou dois programas C funcionalmente equivalentes - o Código Novo 1 (recursivo com aritmética de ponteiros) e o Código Novo 2 (iterativo com pilha manual) - sobre a mesma árvore de 4 nós com limite de profundidade 5. Cada programa foi executado 1.000.000 de vezes consecutivas para garantir estabilidade estatística. A medição utilizou clock_gettime(CLOCK_MONOTONIC) com resolução de nanosegundo, compilação com flag -O2.

1 - Quantidade de Ciclos Processados

Define-se "ciclo" como cada unidade elementar de trabalho: no paradigma recursivo, cada chamada de processarHierarquia(); no iterativo, cada iteração do laço while(). A tabela abaixo detalha a contagem.

Detalhe de Ciclos	Recursivo	Iterativo
Nós visitados por execução	4 nós	11 iterações de loop
Chamadas de função / desvios	4 (stack frames)	11 (while + if)
Overhead de controle de fluxo	Baixo	Alto (pilha manual)
Ciclos totais (1 M execuções)	4.000.000	11.000.000

A função recursiva visita cada nó exatamente uma vez (4 chamadas). O iterativo necessita de 11 iterações porque cada nó é visitado nas duas fases da máquina de estados (fase 0 e fase 1) e há 3 transições adicionais para processar os índices de filhos: $4 \times 2 + 3 = 11$. Resultado: o iterativo processa 175% mais ciclos que o recursivo.

2 - Tempo Total de Execução (Benchmark)

A tabela consolidada abaixo apresenta ciclos e tempos lado a lado para facilitar a análise comparativa.

Paradigma	Recursivo (ponteiros)	Iterativo (pilha manual)
Repetições	1.000.000	1.000.000
Limite de profundidade	5	5
—— 3.1 Ciclos ——	——	——

Ciclos totais processados	4.000.000	11.000.000
Ciclos por execução	4	11
Diferença de ciclos	-	+7.000.000 (+175%)
—— 3.2 Tempo ——	—————	—————
Tempo total (1 M exec.)	28,62 ms	60,33 ms
Tempo por execução	28,62 ns	60,33 ns
Delta de tempo	-	+31,71 ms (ite. mais lento)
Speed-up (rec ÷ ite)	-	0,4744x → rec. 110,8% mais rápido
—— 3.3 Vencedor ——	—————	—————
Performance geral	MELHOR ✓	MAIS LENTO ✗

3 - Conclusão Técnica

Resultado numérico

O paradigma recursivo processou 4.000.000 ciclos em 28,62 ms (28,62 ns/execução). O paradigma iterativo processou 11.000.000 ciclos em 60,33 ms (60,33 ns/execução). O recursivo se mostrou 110,8% mais rápido, com speed-up de 0,4744 em relação ao iterativo (o iterativo levou 2,11× mais tempo para concluir a mesma tarefa).

Análise das causas

Recursivo: Com -O2, o compilador otimiza chamadas de função agressivamente, aproveitando o preditor de desvio e o cache de instruções da call stack nativa. Com apenas 4 nós, o overhead de empilhamento é mínimo.

Iterativo: A pilha manual (vetor ItemPilha) gera acessos adicionais à memória e incrementos de múltiplos contadores (topo, filho_cursor, fase) por iteração. A máquina de estados de duas fases duplica as iterações em relação ao número de nós, produzindo 175% mais ciclos.

Quando cada abordagem é preferível

Recursivo: Recomendado para árvores de profundidade moderada, onde a performance nativa da call stack supera qualquer risco de estouro de pilha. Ideal para o problema proposto.

Iterativo: Indicado em cenários com profundidade potencialmente ilimitada (risco de stack overflow) ou sistemas com pilha restrita. Paga o custo de maior complexidade de código e mais ciclos de CPU em troca de segurança de execução.

Veredicto final

Para o problema proposto, o Código Novo 1 - recursivo com ponteiros - é a implementação de maior performance, sendo 110,8% mais rápido e processando 7.000.000 ciclos a menos que a versão iterativa no total de 1 milhão de execuções. A abordagem iterativa permanece válida como alternativa robusta para cenários de profundidade indefinida, mas não apresenta vantagem de performance na configuração avaliada.

Ambiente: GCC - O2 | clock_gettime(CLOCK_MONOTONIC) | 1.000.000 repetições | Profundidade máxima = 5